ECommerce API - Postman Documentation

📚 Overview

This Postman collection contains complete API documentation for the ECommerce platform, including all endpoints for:

- **Authentication** (Register, Login)
- **Products** (CRUD, filtering by brand/category)
- **Product Variants** (Size, Color, Stock management)
- **Shopping Cart** (Add, Remove, Update, Clear)
- **Categories & Brands** (CRUD operations)
- **Admin Management** (Cart management, user data)

🚀 Getting Started

1. Import the Collection and Environment

1. Open Postman
2. Click **Import** → **Upload Files**
3. Select `ECommerce_API.json` from `/postman/collections/`
4. Select `Development.json` from `/postman/environments/`
5. Set **Development** as the active environment (top-right dropdown)

2. Configure Environment Variables

Edit the **Development** environment with your actual values:

```
baseUrl: http://localhost:7286/api
adminBaseUrl: http://localhost:7286/api/admin
authToken: (Auto-filled after login)
userId: (Auto-filled after login)
```

🔐 Authentication Workflow

1. Register a New User

```
POST /api/auth/register
{
  "userName": "testuser",
  "email": "test@example.com",
  "password": "Test@123"
}
```

Response: `authToken`, `userId`, `refreshToken` (auto-set in environment)

---

2. Login to Get Token

```
POST /api/auth/login
{
  "email": "test@example.com",
  "password": "Test@123"
}
```

Auto-set variables:

- `authToken` → Used for all authenticated requests
- `userId` → Your user ID
- `refreshToken` → Used to refresh expired tokens

---

3. Refresh Access Token

```
POST /api/auth/refresh-token
{
  "refreshToken": "{{refreshToken}}"
}
```

Use when: Access token expires (typically after 15-30 minutes)
Returns: New `authToken` (auto-set)

---

4. Forgot Password (Part 1)

```
POST /api/auth/forgot-password
{
  "email": "test@example.com"
}
```

Effect: Sends OTP code to user's email

---

5. Verify OTP (Part 2)

```
POST /api/auth/verify-otp
{
```

```
  "email": "test@example.com",
  "otp": "123456"
}
```

Effect: Validates OTP code sent to email

---

6. Reset Password (Part 3)

```
POST /api/auth/reset-password
{
  "email": "test@example.com",
  "otp": "123456",
  "newPassword": "NewPassword@123",
  "confirmPassword": "NewPassword@123"
}
```

Effect: Sets new password if OTP is verified

📦 Product Management

Products WITH Variants (e.g., T-Shirt with sizes)

HasVariants = true

```
POST /api/products/create (Admin)
{
  "name": "Premium T-Shirt",
  "description": "High-quality cotton",
  "imageUrl": "https://...",
  "price": 29.99,
  "hasVariants": true,
  "stockQuantity": 0,        // Auto-calculated from variants
  "categoryId": 1,
  "brandId": 1
}
```

Then add variants:

```
POST /api/products/{{productId}}/variants
[
  { "size": "S", "color": "Blue", "stockQuantity": 50 },
  { "size": "M", "color": "Blue", "stockQuantity": 60 },
  { "size": "L", "color": "Blue", "stockQuantity": 40 }
```

```
]
// Product.StockQuantity = 50 + 60 + 40 = 150 (auto-calculated)
```

Products WITHOUT Variants (e.g., Individual posters)

HasVariants = false

```
POST /api/products/create (Admin)
{
  "name": "Movie Poster",
  "hasVariants": false,
  "stockQuantity": 100,      // Direct product-level stock
  ...
}
```

🛒 Shopping Cart Logic

🔘 Important Business Rules

Rule 1: Products WITH Variants

- Must include `productVariantId`
- Stock checked at **variant level**
- Cannot add without selecting size/color

```
POST /api/cart/items
{
  "quantity": 2,
  "productId": 1,
  "productVariantId": 5      // ☑ REQUIRED for HasVariants=true
}
```

Rule 2: Products WITHOUT Variants

- Must NOT include `productVariantId`
- Stock checked at **product level**

```
POST /api/cart/items
{
  "quantity": 1,
  "productId": 2
  // ☑ Leave productVariantId empty
}
```

Cart Price Calculation

When adding to cart, price is **snapshotted**:

```
UnitPrice = Product.Price + ProductVariant.AdditionalPrice (if variant)
```

This ensures cart totals don't change if product prices are updated later.

Cart Summary

```
GET /api/cart/summary

Returns:
{
  "cartId": 1,
  "totalItems": 3,
  "subtotal": 100.00,
  "tax": 14.00,              // 14% tax rate
  "discount": 0,             // Future: coupon system
  "total": 114.00
}
```

🏢 Admin Endpoints

Cart Management (Admin Only)

```
GET /api/admin/carts                           // List all carts
GET /api/admin/carts/{{cartId}}               // Get specific cart
GET /api/admin/carts/user/{{userId}}          // Get user's cart
GET /api/admin/cart-items/cart/{{cartId}}     // List items in cart
DELETE /api/admin/carts/{{cartId}}            // Delete cart
DELETE /api/admin/cart-items/{{cartId}}       // Delete cart item
```

📊 API Response Format

Success Response (200 OK)

```
{
  "status": true,
  "message": "Product created successfully.",
  "data": {
    "id": 1,
```

```
    "name": "T-Shirt",
    ...
  }
}
```

Error Response (400 Bad Request)

```
{
  "status": false,
  "message": "Product not found.",
  "data": null
}
```

Paginated Response

```
{
  "status": true,
  "message": "Products fetched successfully.",
  "data": {
    "items": [...],
    "totalItems": 50,
    "page": 1,
    "pageSize": 10
  }
}
```

🔄 Stock Management Flow

Automatic Stock Calculation (Products WITH Variants)

1. **Create Variant** → Product.HasVariants = true
2. **Update Variant Stock** → Product.StockQuantity auto-recalculated
3. **Delete Last Variant** → Product.HasVariants = false, StockQuantity = 0

Stock Validation

- **Adding to cart**: Check available stock before adding
- **Updating quantity**: Re-validate stock for new quantity
- **Cannot delete variant** if items are in active carts

🔍 Useful Query Parameters

Pagination

```
?page=1&pageSize=10      // Default pagination
```

Filtering

```
GET /api/products/by-category/{{categoryId}}
GET /api/products/by-brand/{{brandId}}
```

📝 Common Workflows

Workflow 1: Create Product with Variants

```
1. POST /api/products/create              → Get productId
2. POST /api/products/{{productId}}/variants (3x)  → Add S, M, L sizes
3. GET /api/products/{{productId}}        → View auto-calculated stock
```

Workflow 2: Customer Shopping

```
1. POST /api/auth/login                   → Get authToken
2. GET /api/products                      → Browse products
3. POST /api/cart/items                   → Add to cart
4. GET /api/cart/summary                  → View total
5. DELETE /api/cart/items/{{id}}          → Remove items
```

Workflow 3: Admin Stock Management

```
1. GET /api/admin/products                → Find product
2. PUT /api/products/{{id}}/variants/{{variantId}}  → Update stock
3. GET /api/admin/carts                   → Check all carts
```

🐛 Troubleshooting

Error: "This product has variants. You must select a variant."

- Solution: For products with `HasVariants=true`, add `productVariantId` to cart request

Error: "This product does not have variants. VariantId must not be provided."

- Solution: For products with `HasVariants=false`, remove `productVariantId` from cart request

Error: "Insufficient stock"

- Check product stock: `GET /api/products/{{productId}}`
- For variants: Check specific variant availability
- Stock includes items already in cart

Error: "Unauthorized" (401)

- Ensure authToken is set in environment
- Re-login to get fresh token
- Check Authorization header is set to Bearer token

📋 Environment Variables

Variable	Purpose	Example
`baseUrl`	API base URL	`http://localhost:7286/api`
`adminBaseUrl`	Admin endpoints	`http://localhost:7286/api/admin`
`authToken`	JWT token (auto-set)	`Bearer eyJhbGc...`
`userId`	Current user ID (auto-set)	`user-123`
`productId`	Product ID for testing	`1`
`variantId`	Variant ID for testing	`1`
`cartId`	Cart ID for admin testing	`1`
`categoryId`	Category ID	`1`
`brandId`	Brand ID	`1`
`page`	Pagination page	`1`
`pageSize`	Items per page	`10`

☑ Best Practices

1. **Always set authToken** in environment after login
2. **Use collection variables** (`{{productId}}`) instead of hardcoding IDs
3. **Test with pagination** for large datasets
4. **Validate stock** before adding to cart in UI
5. **Handle variant selection** for products with multiple options
6. **Cache computed stock** on client for better UX

📞 API Support

For more information, refer to:

- Base URL: `http://localhost:7286`
- Swagger/OpenAPI: `/swagger/index.html` (if available)
- Environment: Development (local testing)

Last Updated: March 1, 2026